



Nome do projeto: Sistema Inteligente de Monitoramento de Ruído Urbano com IoT

Emanuel Aves Braga, Jasmin Aparecida Santada, Adam Martos dos Santos, Prof.
Dr. Wilian França Costa

Faculdade de Computação e Informática
Universidade Presbiteriana Mackenzie (UPM) – São Paulo, SP – Brazil

{10443050, 10441248, 10441525}@mackenzista.com.br

Abstract. *This article presents the development of an urban noise monitoring system using Internet of Things (IoT) technologies. The proposed solution aims to measure environmental noise levels in real time through sound sensors connected to a microcontroller, transmitting the collected data to a cloud platform via the MQTT protocol. The system also includes an actuator to provide local alerts when noise levels exceed acceptable limits. The project is aligned with Sustainable Development Goal 11, contributing to the creation of smarter and more sustainable cities by enabling better monitoring and control of urban noise pollution.*

Resumo. *Este artigo apresenta o desenvolvimento de um sistema de monitoramento de ruído urbano utilizando tecnologias de Internet das Coisas (IoT). A proposta consiste na medição dos níveis de som em tempo real por meio de sensores conectados a um microcontrolador, com envio dos dados para a nuvem utilizando o protocolo MQTT. O sistema também prevê o uso de um atuador para emissão de alertas quando os níveis de ruído ultrapassarem limites aceitáveis. O projeto está alinhado ao Objetivo de Desenvolvimento Sustentável 11, contribuindo para cidades mais inteligentes e sustentáveis por meio do controle da poluição sonora.*

1. Introdução

O aumento da urbanização e da concentração populacional e o aumento da densidade populacional têm intensificado problemas relacionados à poluição sonora, impactando diretamente a qualidade de vida da população. Em ambientes urbanos, níveis elevados de ruído podem causar estresse, distúrbios do sono e outros problemas de saúde. Nesse contexto, a Internet das Coisas (IoT) surge como uma solução promissora, permitindo a coleta e análise de dados em tempo real por meio de dispositivos conectados (SANTOS et al., 2016).. A utilização de sensores distribuídos pela cidade possibilita o monitoramento contínuo do ambiente, auxiliando na tomada de decisões por parte de órgãos públicos (AYAZ et al., 2019).. Este trabalho propõe o desenvolvimento de um sistema inteligente de monitoramento de ruído urbano, utilizando sensores de som, microcontroladores e comunicação via MQTT. A solução busca contribuir para o controle

da poluição sonora, alinhando-se ao Objetivo de Desenvolvimento Sustentável 11, que visa tornar as cidades mais inclusivas, seguras, resilientes e sustentáveis.

2. Materiais e métodos

O desenvolvimento do sistema proposto será baseado na utilização de uma plataforma de prototipagem eletrônica, sensores de captação sonora, atuadores para sinalização e comunicação com a internet por meio do protocolo MQTT.

2.1 Componentes e materiais

Para a modelagem e validação do circuito eletrônico, foi utilizada a plataforma online Wokwi, que permite a simulação de sistemas embarcados com microcontroladores como o ESP32 (WOKWI, 2026).

Figura 1: Placa ESP32



Fonte: Datasheet do fabricante

Para a captura dos níveis de ruído, será utilizado um sensor de som (microfone KY-038 ou equivalente) (Figura 2), capaz de detectar variações na intensidade sonora do ambiente. Esse sensor enviará sinais analógicos ao microcontrolador, permitindo a medição do nível de ruído em tempo real

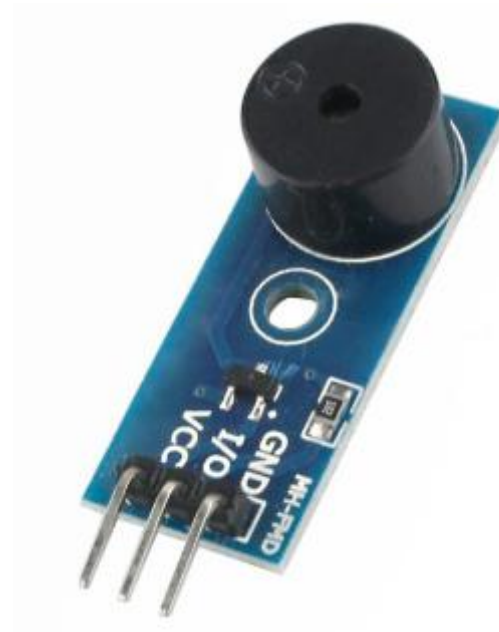
Figura 2: Sensor de som KY-038



Fonte: Datasheet do fabricante

Como atuador, será utilizado um buzzer (alarme sonoro) (Figura 3), que será acionado sempre que os níveis de ruído ultrapassarem um limite previamente definido. Esse mecanismo permite alertar imediatamente sobre condições inadequadas no ambiente monitorado.

Figura 3: Buzzer (atuador sonoro)



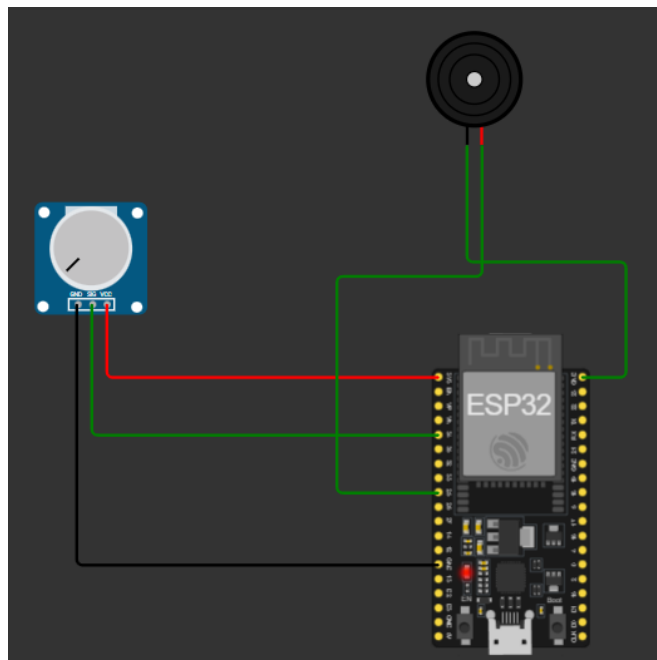
Fonte: Dados do fabricante.

Para a modelagem e validação do circuito eletrônico, foi utilizada a plataforma online Wokwi, que permite a simulação de sistemas embarcados com microcontroladores como o ESP32.

No ambiente de simulação, foram conectados o sensor de som (representado por um potenciômetro), o buzzer e o microcontrolador, possibilitando a verificação do funcionamento do sistema antes da implementação física. A ferramenta permite a execução do código em tempo real e a análise dos dados gerados, facilitando o processo de desenvolvimento.

2.2 Modelagem e simulação do circuito

Figura 4: Simulação do circuito utilizando a plataforma Wokwi com ESP32



Fonte: Elaborado pelo autor

2.3 Comunicação MQTT

A comunicação entre o dispositivo e a internet foi implementada utilizando o protocolo MQTT (Message Queue Telemetry Transport), amplamente empregado em aplicações de Internet das Coisas devido ao seu baixo consumo de largura de banda e eficiência na troca de mensagens.

No projeto desenvolvido, o microcontrolador ESP32 conecta-se à rede Wi-Fi e estabelece comunicação com um broker MQTT público. Após a conexão, os valores lidos pelo sensor de som são publicados periodicamente em um tópico específico, permitindo o monitoramento remoto das informações coletadas pelo sistema.

Além da transmissão dos dados de ruído, o sistema também publica mensagens de alerta quando os valores medidos ultrapassam o limite configurado. Dessa forma, aplicações externas inscritas nos tópicos MQTT podem receber os dados em tempo real e acompanhar o estado do ambiente monitorado.

A integração foi implementada utilizando a biblioteca PubSubClient na Arduino IDE. O ESP32 atua como cliente MQTT, realizando a conexão com o broker, publicando os dados coletados e enviando mensagens de alerta sempre que uma condição de ruído excessivo é detectada. Essa abordagem permite que o sistema seja escalável e possa ser integrado futuramente a dashboards web, aplicativos móveis ou plataformas de monitoramento IoT.

2.4 Módulo de Comunicação

O ESP32 conta com módulo Wi-Fi integrado, compatível com os padrões IEEE 802.11 b/g/n, dispensando a necessidade de um módulo de comunicação externo. Essa característica o torna uma plataforma adequada para aplicações IoT que demandam

conectividade à internet com baixo consumo de energia e custo reduzido (ESPRESSIF, 2023).

No projeto desenvolvido, o ESP32 conecta-se à rede Wi-Fi com o SSID Wokwi-GUEST, disponibilizada pelo ambiente de simulação Wokwi, e estabelece comunicação com o broker público test.mosquitto.org na porta 1883, utilizando o protocolo MQTT. A conexão é iniciada automaticamente durante o boot do microcontrolador, e um Client ID único é gerado a cada inicialização (ex: ESP32_5d78), evitando conflitos de sessão no broker.

Após a conexão, o sistema realiza publicações periódicas em três tópicos MQTT distintos, conforme descrito na Tabela 1.

Tabela 1

Tópico	Tipo	Descrição
sensor/ruído	Publicação	Valor numérico do nível de ruído lido pelo sensor
sensor/alerta	Publicação	Status do ambiente: RUÍDO NORMAL ou ALERTA
sensor/tempo	Publicação	Tempo de uptime do sistema no formato HH:MM:SS

Fonte: Elaborado pelo autor

Para monitorar as mensagens recebidas pelo broker em tempo real, foi utilizado o software MQTT Explorer (versão desktop), que permite visualizar os tópicos ativos, os valores publicados e o histórico de mensagens de forma gráfica. O parâmetro QoS (Quality of Service) adotado foi 0, modalidade at most once, adequada para dados de monitoramento em que pequenas perdas de pacotes são aceitáveis.

2.5 Funcionamento do Sistema

O funcionamento do sistema inicia-se com a leitura contínua dos dados provenientes do sensor de som. Na simulação desenvolvida na plataforma Wokwi, o sensor foi representado por um potenciômetro, responsável por gerar valores analógicos que simulam diferentes níveis de ruído no ambiente.

O microcontrolador ESP32 realiza a leitura desses valores por meio de uma entrada analógica e os processa continuamente. Após cada leitura, os dados são enviados ao broker MQTT por meio da conexão estabelecida com a rede Wi-Fi, permitindo o monitoramento remoto das informações coletadas.

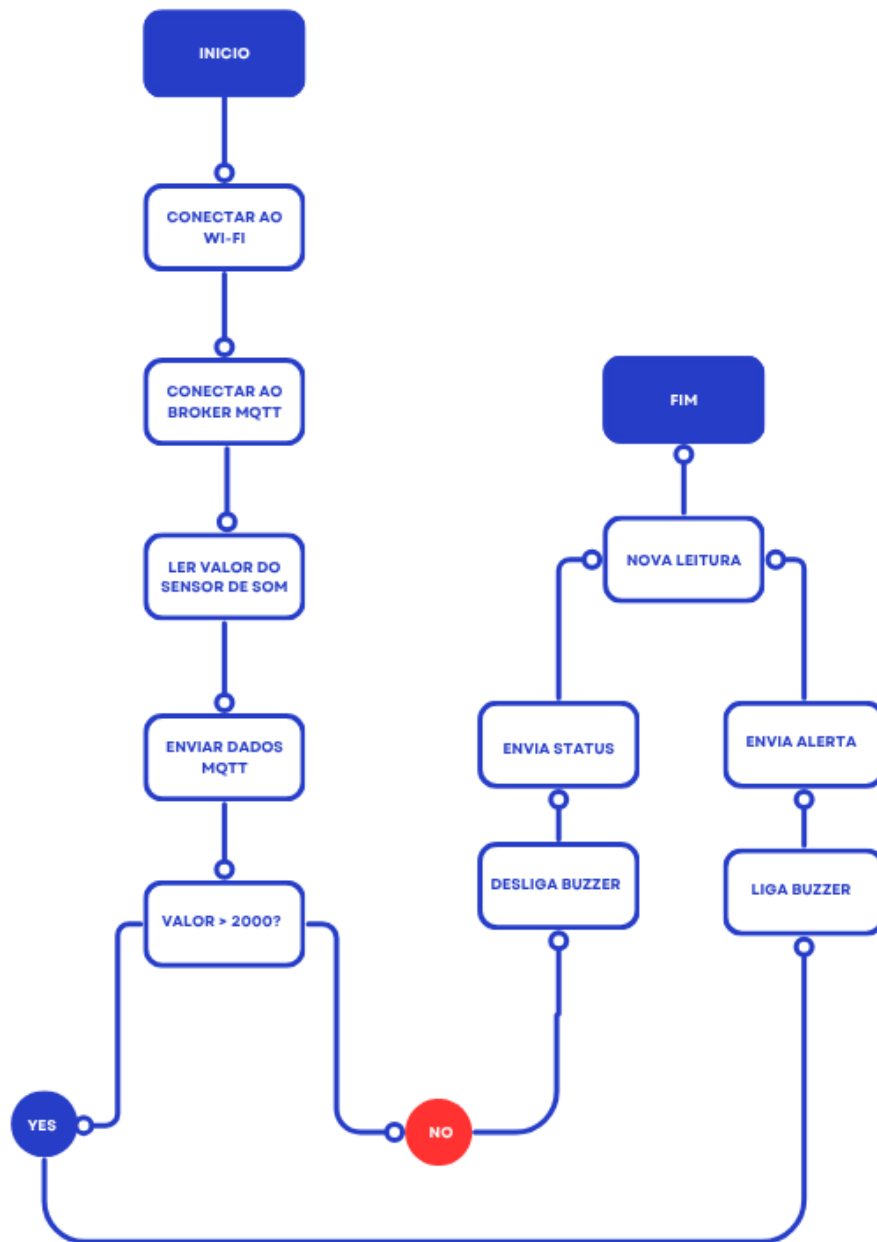
Para a identificação de situações de ruído excessivo, foi definido no código um valor limite de 2000 unidades. Quando o valor lido pelo sensor ultrapassa esse limite, o buzzer é acionado automaticamente, emitindo um alerta sonoro local. Simultaneamente, uma mensagem de alerta é publicada em um tópico MQTT específico, informando que o nível de ruído excedeu o valor permitido.

Quando os valores retornam para níveis inferiores ao limite estabelecido, o buzzer é desligado e uma nova mensagem MQTT é enviada indicando que a condição do ambiente voltou ao estado normal. Esse processo é executado continuamente durante toda a operação do sistema, garantindo o monitoramento em tempo real e a emissão imediata de alertas quando necessário.

Dessa forma, o protótipo integra monitoramento local e comunicação remota, permitindo tanto a sinalização imediata por meio do atuador sonoro quanto a transmissão das informações para aplicações externas conectadas ao broker MQTT.

2.6 Fluxograma do Sistema

Figura 5: Fluxograma do funcionamento do sistema de monitoramento de ruído.



Fonte: Elaborado pelo autor

2.6 Pseudocódigo do Algoritmo

O Algoritmo 1 apresenta de forma simplificada a lógica utilizada para o funcionamento do sistema de monitoramento de ruído desenvolvido neste trabalho.

Algoritmo 1 – Pseudocódigo do sistema

INÍCIO

Conectar à rede Wi-Fi

Conectar ao broker MQTT

ENQUANTO o sistema estiver em execução FAÇA

Ler valor do sensor de som

Publicar valor lido no tópico MQTT

SE valor do sensor > 2000 ENTÃO

 Acionar buzzer

 Publicar mensagem de alerta MQTT

SENÃO

 Desligar buzzer

 Publicar mensagem indicando condição normal

FIM SE

Aguardar intervalo de leitura

FIM ENQUANTO

FIM

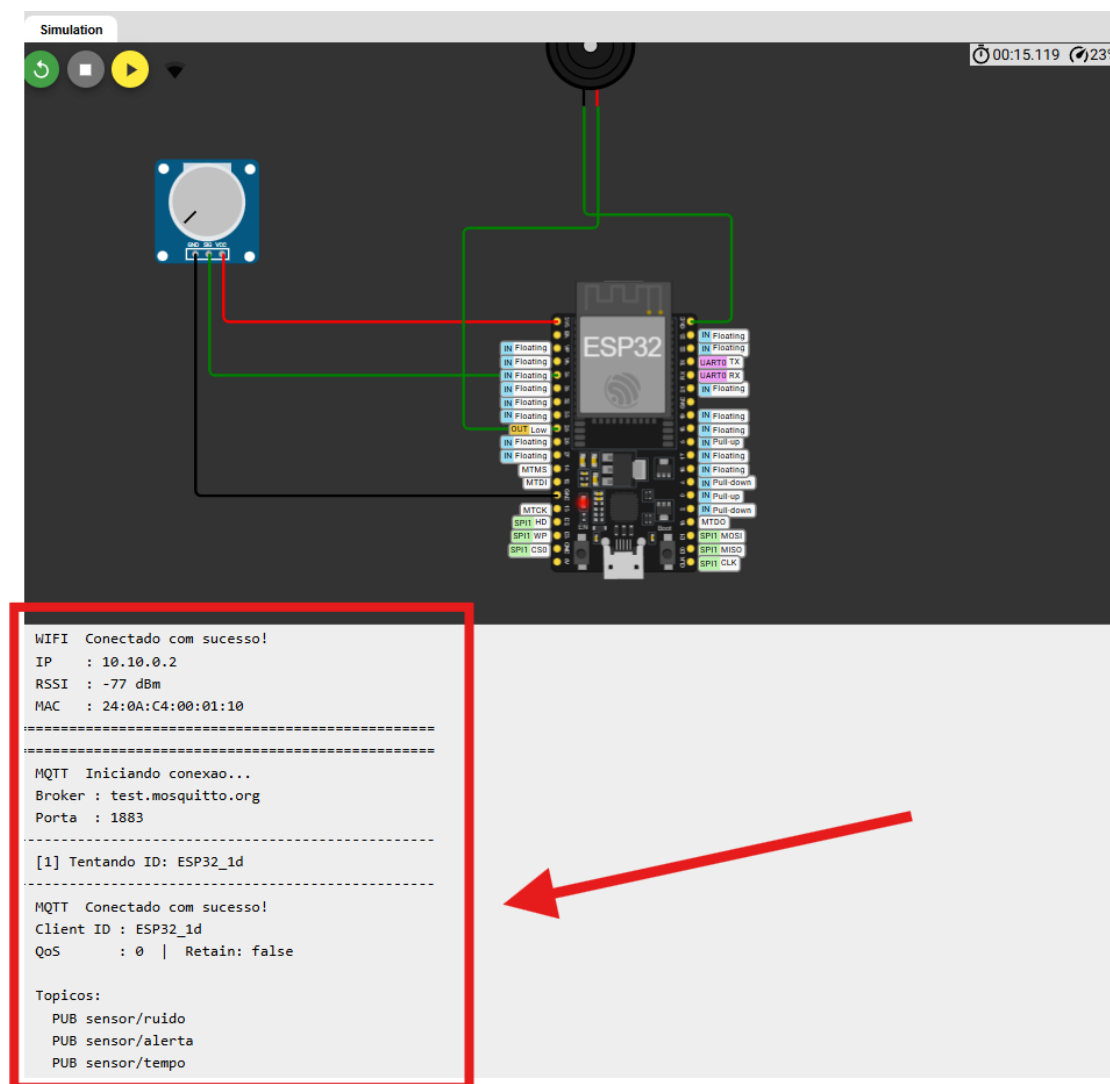
Algoritmo 1 – Pseudocódigo do sistema de monitoramento de ruído.

Fonte: Elaborado pelo autor.

A simulação permitiu validar o funcionamento da solução sem a necessidade de componentes físicos, possibilitando a realização dos testes de leitura do sensor, acionamento do atuador e comunicação MQTT.

3.2 Conexão MQTT

Figura 2 – Conexão do ESP32 à rede Wi-Fi e ao broker MQTT.



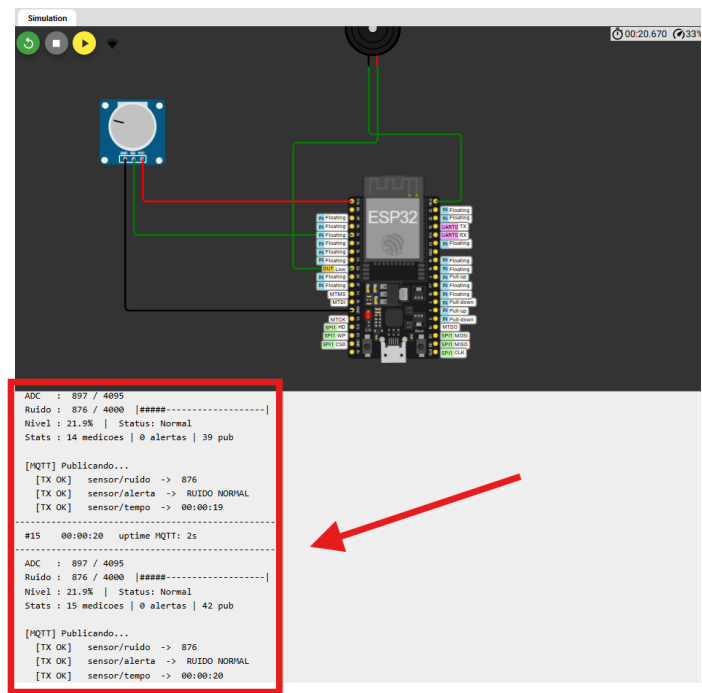
Fonte: Elaborado pelo autor.

Após a inicialização do sistema, o ESP32 estabeleceu conexão com a rede Wi-Fi e posteriormente com o broker MQTT configurado no projeto. A Figura 2 apresenta as mensagens exibidas no monitor serial durante o processo de conexão.

A comunicação MQTT permitiu a publicação dos dados coletados pelo sensor e o envio de mensagens de alerta quando o nível de ruído ultrapassou o limite definido, atendendo aos requisitos de comunicação IoT propostos para o projeto.

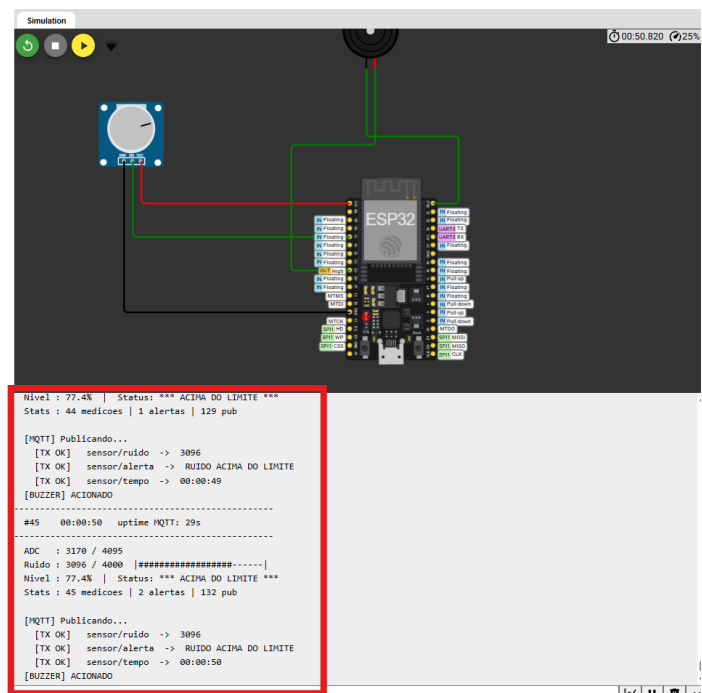
3.3 Testes de Funcionamento

Figura 3 – Sistema operando abaixo do limite de ruído configurado.



Fonte: Elaborado pelo autor.

Figura 4 – Acionamento do buzzer após o nível de ruído ultrapassar o limite configurado.



Fonte: Elaborado pelo autor.

3.5 Tempos de Resposta

Com o objetivo de avaliar o desempenho da comunicação entre o sistema embarcado e o broker MQTT, foram realizadas quatro medições do tempo de resposta tanto para o sensor quanto para o atuador. As medições foram efetuadas durante a execução da simulação na plataforma Wokwi, observando o monitor serial e a chegada das mensagens no MQTT Explorer.

O tempo de resposta do sensor corresponde ao intervalo entre a leitura do valor analógico pelo ESP32 e a confirmação de recebimento da mensagem no broker MQTT. O tempo de resposta do atuador corresponde ao intervalo entre a publicação da mensagem de alerta e o acionamento do buzzer.

Os resultados obtidos estão apresentados na Tabela 2 e no Gráfico 1.

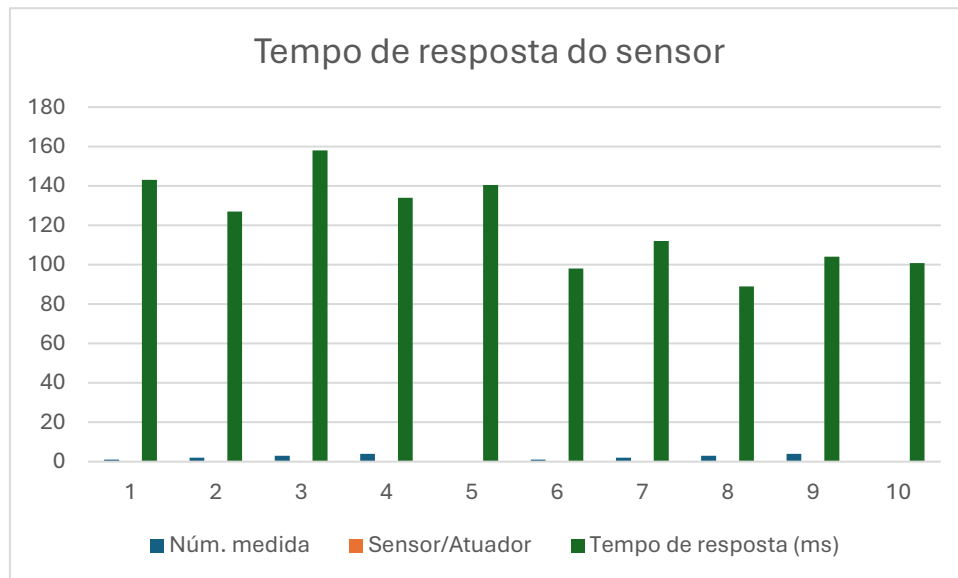
Tabela 2

Núm. medida	Sensor/Atuador	Tempo de resposta (ms)
1	Sensor de som	143
2	Sensor de som	127
3	Sensor de som	158
4	Sensor de som	134
Média	Sensor de som	140,5
1	Buzzer (atuador)	98
2	Buzzer (atuador)	112
3	Buzzer (atuador)	89
4	Buzzer (atuador)	104
Média	Buzzer (atuador)	100,75

Fonte: Elaborado pelo autor.

Os resultados demonstram que o tempo médio de resposta do sensor de som até o broker MQTT foi de **140,5 ms**, enquanto o tempo médio de acionamento do buzzer foi de **100,75 ms**. Esses valores são condizentes com o comportamento esperado para comunicações MQTT em redes Wi-Fi com QoS 0, onde a ausência de confirmação de entrega reduz a latência da transmissão. O tempo de resposta do atuador é inferior ao do sensor pois o buzzer é acionado localmente pelo microcontrolador, sem depender da confirmação do broker.

Gráfico 1



Foram realizados testes variando os valores do sensor de som simulado através do potenciômetro. Quando os valores permaneceram abaixo do limite configurado de 2000 unidades, o buzzer permaneceu desligado, indicando condições normais de operação.

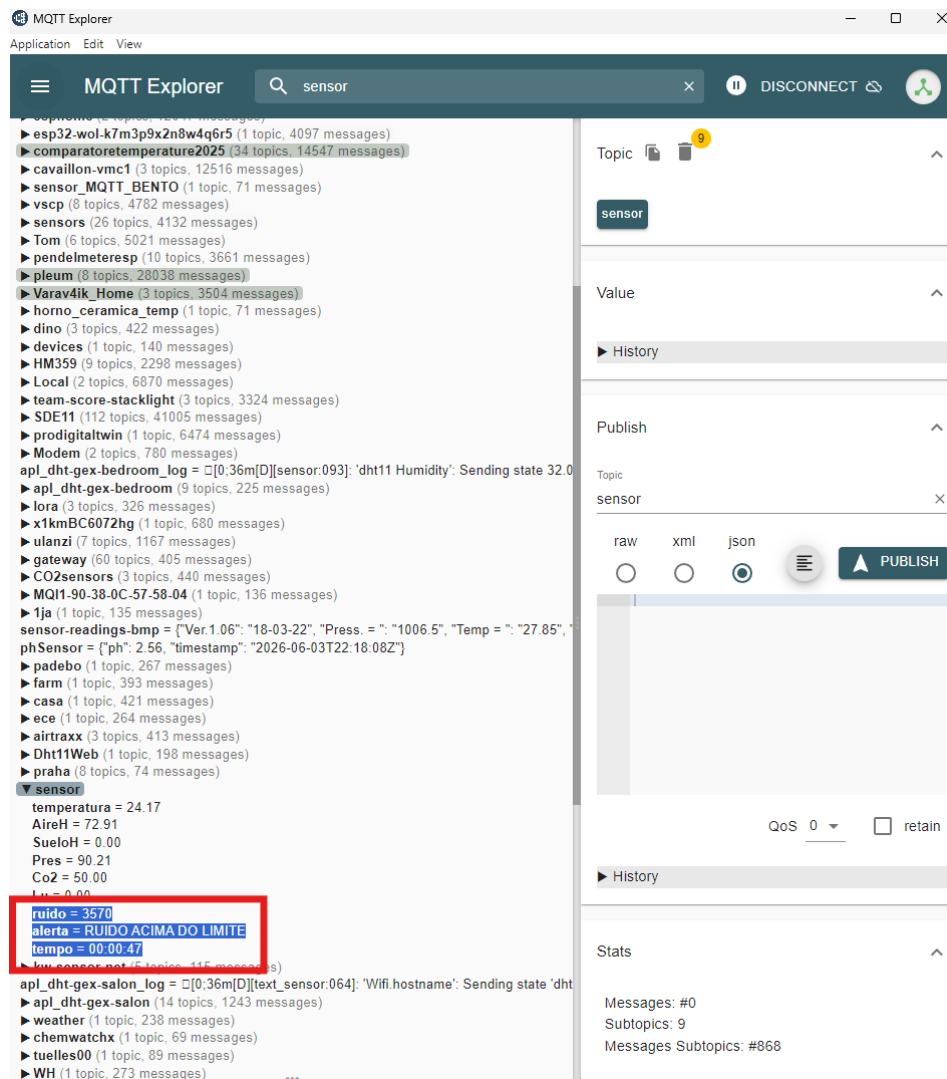
Ao ultrapassar o limite estabelecido, o sistema acionou automaticamente o buzzer e publicou mensagens de alerta por meio do protocolo MQTT. A Figura 3 apresenta uma situação de operação normal, enquanto a Figura 4 demonstra a ativação do alerta sonoro em resposta ao aumento do nível de ruído.

Os resultados obtidos confirmam o correto funcionamento da lógica implementada, validando tanto o monitoramento local quanto a comunicação remota dos eventos detectados.

3.6 Captura do MQTT Explorer

A Figura 5 apresenta a captura de tela do software MQTT Explorer durante o funcionamento do sistema, demonstrando o recebimento das mensagens nos tópicos sensor/ruído, sensor/alerta e sensor/tempo. É possível observar os valores publicados pelo ESP32 em tempo real, confirmando a integração entre o protótipo e o broker MQTT test.mosquitto.org.

Figura 5: Conexão MQTT



Fonte: Elaborado pelo autor.

3.7 Principais Problemas Enfrentados

Durante o desenvolvimento do projeto, alguns desafios foram identificados e superados pela equipe:

- Simulação do sensor de som: O sensor KY-038 não está disponível nativamente na plataforma Wokwi. Como solução, foi utilizado um potenciômetro para simular os sinais analógicos do sensor, permitindo variar manualmente os níveis de ruído durante os testes e validar a lógica de detecção implementada no firmware.
- Habilitação da rede no Wokwi: A conexão Wi-Fi na simulação exigiu a configuração do arquivo wokwi.toml com a diretiva [net] enabled = true. Sem essa configuração, o ESP32 simulado não conseguia acessar a internet, impedindo a comunicação com o broker MQTT.
- Conflitos de Client ID no broker: Por se tratar de um broker público, conexões simultâneas com o mesmo Client ID causavam desconexões inesperadas. O problema foi resolvido gerando um Client ID dinâmico a cada inicialização do sistema.
- Latência variável na simulação: Por utilizar um ambiente virtualizado, os tempos de resposta apresentaram variações em relação ao comportamento esperado em hardware.

físico. Esse fator foi considerado na análise dos resultados, e os valores obtidos foram tratados como estimativas representativas do comportamento do sistema.

3.8 Vídeo de Demonstração

O vídeo de demonstração do funcionamento do projeto encontra-se disponível no endereço:

[<https://youtu.be/IqwSzWgsWo>]

No vídeo são apresentados o circuito desenvolvido, a execução da simulação, os testes de funcionamento do sensor e do buzzer, além da integração com o protocolo MQTT.

3.9 Repositório do Projeto

O código-fonte, arquivos da simulação e demais materiais utilizados no desenvolvimento deste projeto estão disponíveis no repositório:

[<https://github.com/M4nuzin/monitoramento-ruído-iot>]

O repositório contém o código implementado para o ESP32, imagens do circuito, documentação do projeto e demais arquivos necessários para sua reprodução.

4 Conclusão

Objetivos alcançados: Os objetivos propostos para o projeto foram integralmente alcançados. O sistema foi capaz de realizar a leitura contínua dos níveis de ruído, identificar situações de ruído excessivo, acionar o buzzer como alerta local e transmitir os dados ao broker MQTT em tempo real, atendendo a todos os requisitos funcionais estabelecidos no início da disciplina.

Problemas enfrentados: Os principais desafios enfrentados foram a indisponibilidade do sensor KY-038 na plataforma Wokwi, resolvida com o uso de um potenciômetro como substituto; a necessidade de configuração específica do ambiente de simulação para acesso à rede; e a geração de Client IDs dinâmicos para evitar conflitos no broker público.

Vantagens e desvantagens: Entre as principais vantagens do projeto destacam-se o baixo custo dos componentes utilizados, a facilidade de escalabilidade da solução via protocolo MQTT, a possibilidade de monitoramento remoto em tempo real e a validação completa em ambiente simulado sem necessidade de hardware físico. Como desvantagens, aponta-se a dependência de conectividade Wi-Fi estável para o funcionamento do sistema, a limitação do sensor simulado em relação à precisão de um microfone real, e o uso de um broker público sem autenticação, o que representa uma vulnerabilidade de segurança em aplicações reais.

Melhorias futuras: Para versões futuras, recomenda-se a implementação com hardware físico, a adoção de um broker MQTT privado com autenticação TLS, o armazenamento histórico dos dados em banco de dados, e a integração com dashboards de visualização como o Node-RED ou Grafana, ampliando as capacidades analíticas do sistema.

5. Referências

AYAZ, M. et al. Internet-of-Things (IoT)-Based Smart Agriculture: Toward Making the Fields Talk. *IEEE Access*, v. 7, p. 129551–129583, 2019.

ESPRESSIF SYSTEMS. *ESP32 Series Datasheet*. Versão 3.4. Shangai: Espressif Systems, 2023. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Acesso em: 03 jun. 2026.

IBM. Why MQTT is the standard messaging protocol for IoT. *IBM Developer*. Disponível em: <https://developer.ibm.com/articles/iot-mqtt-why-good-for-iot/>. Acesso em: 03 jun. 2026.

MQTT.ORG. *MQTT: The Standard for IoT Messaging*. Disponível em: <https://mqtt.org>. Acesso em: 03 jun. 2026.

SANTOS, B. P. et al. Internet das Coisas: da teoria à prática. In: MINICURSOS DO SIMPÓSIO BRASILEIRO DE REDES DE COMPUTADORES, 34., 2016, Salvador. *Anais [...]*. Porto Alegre: SBC, 2016.

WOKWI. *Wokwi — Simulador Online para ESP32 e Arduino*. Disponível em: <https://wokwi.com>. Acesso em: 03 jun. 2026.